Ayame: Fast Logging for Serial Safety Net

Shun Sasaki[1,a)] Takayuki Tanabe[1,b)] Hideyuki Kawashima[2,c)]

Abstract: Concurrency control is essential for database access, and the Serial Safety Net (SSN) represents one of the state-of-the-art methods in this field. Integrating SSN with logging ensures that transaction processing is recoverable. While P-WAL is an efficient logging method that leverages parallel I/O extensively, it still suffers from certain limitations. To overcome these drawbacks, this paper proposes a novel logging method, Ayame. Ayame incorporates three techniques: reusing SSN commit timestamps as logical LSNs, separating workers, flushers, and committers, and dependency-closed durable acknowledgment. Experimental results demonstrate that Ayame achieves a 4.4-fold performance improvement over P-WAL in YCSB-A at 32 worker threads.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR parallelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; and P-WAL partitions the log into multiple streams [10], [11], [12]. Although these systems differ in the target storage device, log format, and recovery procedure, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing. At the same time, strong multiversion protocols such as SI, SSI, and SSN introduce timestamp and dependency structures that are not fully reflected in conventional parallel logging designs. The motivation of this work is therefore to clarify how P-WAL can be combined with timestamp-based CC protocols, especially SSN, without unnecessarily reintroducing global ordering and durable-prefix bottlenecks.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while it still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [13]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-

[1] Graduate School of Media and Governance, Keio University, Japan
[2] Faculty of Environment and Information Studies, Keio University, Japan
a) shun.sasaki@keio.jp
b) tanab@keio.jp
c) river@sfc.keio.ac.jp

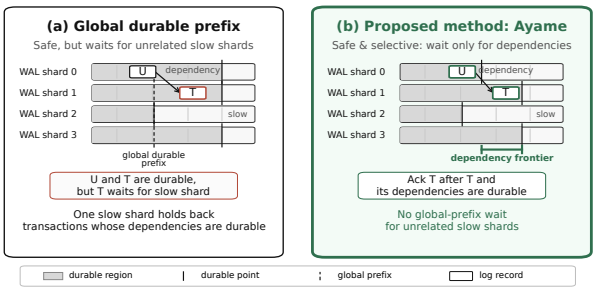

Fig. 1 Durable acknowledgment in parallel WAL. Global-prefix acknowledgment waits for the slowest shard even when a transaction does not depend on it. Ayame instead waits only for the shard positions in the transaction's dependency frontier.

prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

P3. Global-prefix over-waiting: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

In asynchronous durability protocols, logical commits and durable commit acknowledgments must be distinguished. A worker may continue after enqueueing a log record, but the transaction is not durable from the client's perspective until the system returns a durable acknowledgment. Therefore, this paper uses *durable-acknowledgment throughput* as the main throughput metric and reports it together with pending durable commits and tail latency.

1.3 Approach

This paper proposes *Ayame*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. Ayame targets timestamp-based concurrency control such as SSN, which already computes a logical commit order. Its goal is not merely to maximize raw logical commit throughput, but to return durable commit acknowledgments safely without unnecessary global-prefix waiting.

Ayame addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. Ayame reuses SSN's commit timestamp (cstamp) as the logical LSN for recovery. WAL shards still maintain local physical positions, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and eliminates WAL-side separate global LSN allocation on the commit path.

A2. Worker/flusher/committer separation. Ayame decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatasync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. Ayame replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, Ayame maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery safety while avoiding waits for unrelated WAL shards.

We implement Ayame on CCBench and evaluate it against P-WAL using SSN and YCSB workloads. The detailed experimental methodology, figures, and tables are presented in Section 4.

1.4 Organization

The rest of this paper is organized as follows. Section 2 describes transaction processing, including serial-safety-net and parallel logging protocols. Section 3 presents Ayame, which is a fast logging protocol. Section 4 describes the evaluation of Ayame compared with the P-WAL protocol using SSN. Section 5 discusses related work. Section 6 finally concludes this paper.

2. Preliminaries

2.1 Concurrency Control
2.2 Timestamp-based Protocols
2.2.1 Serial Safety Net
2.3 Write Ahead Logging for Recovery
2.4 Logging
2.4.1 P-WAL

P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams, which we hereafter call *WAL shards*. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can

Algorithm 1 Per-Thread WAL Commit for Update Transactions (P-WAL)

```
1: Shared: global LSN counter L; per-stream durable LSN f[i]; global durable prefix P = min_i f[i]
2: function PwalCommit(T, worker w)
3:     records ← BuildLogRecords(T)                ▷ writes and commit record
4:     for all r ∈ records do
5:         r.lsn ← FetchInc(L)                     ▷ global LSN allocation
6:         Append(stream[w], r)                    ▷ worker-private WAL stream
7:     end for
8:     T.ℓ ← max{ r.lsn | r ∈ records }            ▷ commit LSN
9:     Write(stream[w]); Fdatasync(stream[w])       ▷ worker waits for persistence
10:    f[w] ← max(f[w], T.ℓ)
11:    P ← min_i f[i]                              ▷ global durable prefix
12:    while P < T.ℓ do
13:        P ← min_i f[i]
14:    end while
15:    Acknowledge(T)
16: end function
```

spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

P3. Global-prefix over-waiting: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

3. Proposal: Ayame

Ayame is a logging method that integrates durable write-ahead logging into SSN while avoiding the three problems of P-WAL identified in Section 2. It combines three techniques. First, it reuses the commit timestamp that SSN already assigns as the logical log sequence number, eliminating the separate WAL-side global counter (addresses **P1**). Second, it splits commit processing into worker, flusher, and committer roles so that worker threads do not block on `fdatasync` or durable-ack waiting, subject only to a bounded backpressure limit (addresses **P2**). Third, it acknowledges a transaction using a *dependency-closed durable frontier* instead of a global durable prefix, so that a transaction waits only for the log records it actually depends on (addresses **P3**).

We use the following notation. A frontier F is a vector of per-shard log positions. $F \sqcup F'$ denotes the element-wise maximum, $F \leq F'$ holds iff $F[i] \leq F'[i]$ for every shard i, and $\perp$ denotes the empty frontier whose positions are all zero. D denotes the durable vector, where $D[i]$ is the largest log position of shard i that is known to be durable; each $D[i]$ advances monotonically.

3.1 Commit Timestamp as Logical LSN

In SSN, a committing transaction T is assigned a commit timestamp $T.c$ (its *cstamp*) that fixes its position in the serialization order. We assume that SSN assigns a unique commit timestamp to each committed update transaction, so these timestamps form a total order that recovery uses as the replay order. A separate WAL-side global LSN, as used by P-WAL, would redundantly re-encode the same logical order and add a second global-counter access to every commit. Ayame instead reuses $T.c$ directly as the logical LSN that orders recovery replay. No WAL-side global sequence number is allocated; in our evaluation the per-transaction WAL-side global-counter access drops to zero.

Removing the global *logical* LSN does not remove physical log addressing. Each WAL shard keeps a local position (a per-shard sequence number and byte offset) that locates a record within its log file. What Ayame eliminates is the duplicated global ordering counter, not the per-shard physical position.

3.2 Worker, Flusher, and Committer Separation

Ayame divides commit processing into three roles. A *worker* executes the transaction, validates it with SSN, assigns the commit timestamp, appends the log record to its own WAL shard, publishes durability metadata, and registers a durable-acknowledgment request. The worker then returns to its next transaction; it does not call `fdatasync` and does not wait for a durable acknowledgment. A *flusher* owns one WAL shard, batches the queued records, writes them, calls `fdatasync`, and advances the shard's durable position. A *committer* returns durable acknowledgments to clients once the durable condition of a registered transaction is met. Workers therefore do not block on storage synchronization or durable-ack waiting, and keep executing transactions at full concurrency even though every update commit must still be made durable; the cost of making the log durable is paid by the flusher and committer. The one exception is a bounded backpressure limit: to cap memory use and acknowledgment latency, a worker stalls only if the number of registered but not-yet-acknowledged durable commits reaches a configured maximum. Accordingly, we report throughput as the number of durable acknowledgments returned to clients, not the number of logical commits.

Algorithm 2 shows the worker-side commit protocol. The commit timestamp $T.c$ assigned for SSN validation doubles as the logical LSN (line 2). After validation, the worker reserves its log position on its shard (line 11) and forms the closed frontier F by extending its dependency frontier $T.dep$ with that position (line 12). It then builds the log record, whose commit record carries F (line 13), and enqueues it without flushing or waiting (line 14). Ayame installs F as the write frontier of the versions T created (line 16) and

Algorithm 2 Worker-Side Commit Protocol (Ayame)

```
1: function COMMIT(T)
2:     T.c ← commit timestamp (cstamp) assigned by SSN ▷ logical LSN
3:     if SSNVALIDATE(T) fails then
4:         abort T; return
5:     end if
6:     if T's write set is empty then ▷ read-only
7:         ACK(T) ▷ read-only has no recoverable effect
8:         return logical read-only commit
9:     end if
10:    s ← WAL shard assigned to this worker
11:    T.seq ← RESERVESEQ(wal[s]) ▷ reserve this transaction's log position
12:    F ← T.dep;  F[s] ← max(F[s], T.seq) ▷ closed frontier
13:    r ← BUILDRECORD(T.c, F, T.write_set) ▷ commit record carries F
14:    ENQUEUE(wal[s], r) ▷ append only; no flush, no wait
15:    for all versions v created by T do
16:        v.wf ← F
17:    end for
18:    PUBLISHVERSIONS(T.write_set) ▷ visible only after wf is installed
19:    for all versions v read by T do
20:        v.rf ← v.rf ⊔ F ▷ monotone merge
21:    end for
22:    REGISTER(F, T) ▷ durable ack deferred to the committer
23:    return logical commit
24: end function
```

Algorithm 3 Dependency Collection (Ayame)

```
1: function BEGIN(T)
2:     T.dep ← ⊥
3: end function

4: function READVERSION(T, v)
5:     add v to T's read set
6:     if T is known read-only then
7:         return ▷ SI read-only fast path
8:     end if
9:     T.dep ← T.dep ⊔ v.wf ▷ the writer of v
10: end function

11: function OVERWRITEVERSION(T, v)
12:     add the new version to T's write set
13:     T.dep ← T.dep ⊔ v.wf ⊔ v.rf ▷ the writer of v and its readers
14: end function
```

publishes those versions (line 18); a version becomes visible only after its write frontier is installed, so any later reader or overwriter observes the writer's complete frontier. It also merges F into the read frontiers of the versions T read (line 20). Finally, the worker registers F for durable acknowledgment (line 22) before returning a logical commit.

The flusher does not require an algorithm of its own. Each flusher owns one WAL shard, repeatedly collects the records queued by workers, writes them as a single batch, calls `fdatasync` once for the batch, and then advances the shard's durable position by invoking ONDURABLEADVANCE (Algorithm 4). Batching many records per `fdatasync` is what raises the number of commits made durable per synchronization and reduces flush pressure relative to P-WAL's per-transaction synchronization.

3.3 Dependency-Closed Durable Acknowledgment

The remaining question is when a logically committed transaction may be acknowledged as durable. Ayame answers this with a frontier attached to each version. A version carries a *write frontier* wf, the closed frontier of the transaction that created it. A writer publishes wf before its version becomes visible, so any transaction that later reads or overwrites the version observes the complete write frontier of its producer. During execution, a transaction joins these write frontiers into its dependency frontier $T.dep$, as shown in Algorithm 3: reading a version makes T depend on the writer of that version (line 9), and overwriting a version makes T depend on the writer of the version it supersedes (line 13). A version also carries a *read frontier* rf, into which readers monotonically merge their closed frontiers; OVERWRITEVERSION additionally joins rf so that an overwriter conservatively tracks the readers of the version it supersedes. As discussed below, this last term is an anti-dependency over-approximation that the safety argument does not rely on. Transactions known to be read-only skip WAL generation and durable-acknowledgment registration. When the read-only fast path applies, they also avoid dependency-frontier collection; otherwise, only lightweight read-path frontier bookkeeping may remain.

At commit, the closed frontier F (line 12 of Algorithm 2) extends $T.dep$ with T's own log position. By induction over the dependency order, F therefore contains the log positions of the transitive closure of the read-from and version-order (write-write) dependencies of T, all of which are recorded through write frontiers. Publishing F as the write frontier of the versions written by T lets future readers and overwriters inherit T's closed frontier through Algorithm 3. Merging F into the read frontier of the versions read by T conservatively delays future overwriters of those versions, but this read-frontier term is not required for the safety proof.

Algorithm 4 shows the committer side. REGISTER parks T on each shard whose durable position has not yet reached the requirement of F (line 5); if no shard is behind, T is acknowledged immediately (line 8). ONDURABLEADVANCE, invoked by a flusher after `fdatasync`, advances the durable position of one shard and wakes the parked transactions whose requirement on that shard is now satisfied, releasing the acknowledgment once a transaction's last outstanding shard becomes durable (line 17). REGISTER and ONDURABLEADVANCE execute under a single lock so that a durable advance cannot slip between the durability test and the parking step.

The acknowledgment rule is exactly $F \leq D$: a transaction is acknowledged once its own record and every record it tran-

Algorithm 4 Durable Acknowledgment (Ayame)

```
    State: durable vector D; per-shard waitlists W[i] ordered by
    required position
 1: function Register(F, T)                          ▷ atomic w.r.t.
    OnDurableAdvance
 2:     T.remaining ← 0
 3:     for all shards i such that F[i] > D[i] do
 4:         T.remaining ← T.remaining + 1
 5:         insert (F[i], T) into W[i]
 6:     end for
 7:     if T.remaining = 0 then
 8:         Ack(T)        ▷ own record and all dependencies durable
 9:     end if
10: end function

11: function OnDurableAdvance(i, d)          ▷ called by flusher i
    after fdatasync
12:     D[i] ← d
13:     while W[i] is not empty and MinKey(W[i]) ≤ d do
14:         (need, T) ← PopMin(W[i])
15:         T.remaining ← T.remaining − 1
16:         if T.remaining = 0 then
17:             Ack(T)
18:         end if
19:     end while
20: end function
```

sitively depends on are durable. We now make the safety of this rule precise. For a committed update transaction U, let s_U and $U.seq$ be the shard and position of its log record, so its published closed frontier F_U satisfies $F_U[s_U] \geq U.seq$. Let $\mathrm{cl}(T)$ be the set of update transactions reachable from T through read-from dependencies (T reads a version written by U) and version-order dependencies (T overwrites a version written by U), including T itself. Both kinds of dependency are recorded by joining the relevant version's write frontier, which its writer publishes before the version becomes visible.

Lemma 1 (Dependency-closed durability) If $\textsc{Ack}(T)$ is invoked, then for every update transaction $U \in \mathrm{cl}(T)$ the WAL record of U is durable.

Proof. Dependency collection (Algorithm 3) joins the write frontier of each version T reads or overwrites into $T.dep$, and the commit protocol sets $F = T.dep$ with $F[s_T] \geq T.seq$ (line 12). A writer publishes its write frontier before its version becomes visible, so T observes the complete frontier F_U of every U it directly depends on; since each F_U is itself closed, induction over the dependency order gives $F \geq F_U$, hence $F[s_U] \geq F_U[s_U] \geq U.seq$, for every $U \in \mathrm{cl}(T)$. $\textsc{Ack}(T)$ is invoked only when $F \leq D$ (Algorithm 4), so $D[s_U] \geq U.seq$. Since $D[i]$ is a durable prefix of shard i, the record of U is durable.

Each commit record stores the transaction's commit timestamp $T.c$ and its closed dependency frontier F, alongside the redo records that carry the write set. During recovery, Ayame reconstructs the durable vector D by parsing each shard's records in order and advancing $D[i]$ until the first record that does not parse as a complete, well-formed entry, which indicates a torn final write; records beyond that point are ignored. A distinguished commit marker identifies each transaction's commit record. Ayame then collects the committed transactions whose commit records lie within D, keeps only those whose stored frontier is covered by the durable vector (that is, $F \leq D$), and replays this dependency-closed set in increasing cstamp order. Because the replayed set is closed under $F \leq D$, a recovered transaction never appears without the transactions it depends on, even if its own commit record happened to be durable while a dependency's was not.

Theorem 1 (Recoverable acknowledgment) Suppose a crash occurs and recovery replays exactly the durable WAL records in logical-LSN order. If T was acknowledged before the crash, recovery never yields a state that contains the effects of T but omits the effects of some $U \in \mathrm{cl}(T)$.

Proof. By Lemma 1, every $U \in \mathrm{cl}(T)$ has a durable record and is therefore replayed. Each dependency edge $U \to T$ is a serialization edge of SSN, so U's commit timestamp precedes T's; because logical LSNs equal commit timestamps, replay in logical-LSN order installs every such U before T. Hence whenever T's effects are installed, the effects of all $U \in \mathrm{cl}(T)$ are already present.

The lemma and theorem rest only on write frontiers, which are published before a version is visible and are therefore observed completely by every reader and overwriter. The read-frontier term that OverwriteVersion joins (the *rf* of an overwritten version) makes acknowledgment additionally wait for the readers of that version, an anti-dependency. This term is a conservative overapproximation: it can only delay an acknowledgment, never release one early, and the safety argument above does not use it. Because readers publish *rf* concurrently with overwriters, it is a best-effort addition rather than a guaranteed bound, which is why we keep the proof over write frontiers alone.

The rule is also less conservative than a global durable prefix: a transaction waits only on the shards that appear in its frontier, so an unrelated slow shard never delays it.

4. Evaluation

4.1 Implementation

We implement Single WAL, P-WAL, and Ayame on CCBench [7], a concurrency-control benchmarking framework with minimal functionality for protocol analysis, using Masstree as the index and `mimalloc` as the allocator. The code is written in C++20 and compiled with GCC at the `-O3` optimization level. Our implementation is publicly available.[*1]

A key methodological point is that Single WAL, P-WAL, and Ayame are implemented in the *same* binary and selected at run time by environment flags (`CCBENCH_WAL_MODE` and `CCBENCH_WAL_DURABLE_MODE`). The CCBench SI+SSN

[*1] The source code is available in our public repository for reproduction.

protocol implementation, the index, the workload generator, and the pre-commit path are therefore identical across the three systems; only the WAL durability path differs. Single WAL appends to one shared log under a global latch and synchronizes it per commit. P-WAL gives each worker a private WAL shard and synchronizes it per commit, acknowledging through a global durable prefix. Ayame uses the worker/flusher/committer pipeline of Section 3 with dependency-closed acknowledgment. Each WAL redo record carries the key, value, storage identifier, and operation type of a write; the terminating commit record additionally stores the transaction's commit timestamp and its closed dependency frontier, which recovery uses to admit only the dependency-closed set of durable transactions. Read-only WAL skipping is enabled for all three systems; once a transaction is known to be read-only it produces no WAL record. For read-only transactions Ayame also skips durable-acknowledgment registration, but depending on the read path it may still execute lightweight frontier bookkeeping; YCSB-C quantifies this residual overhead.

Following the worker/flusher/committer separation of Section 3, Ayame offloads durability to background threads: it uses 1, 1, 1, 2, 4, 7, 9, and 19 flusher threads for 1, 2, 4, 8, 16, 32, 48, and 96 worker threads, respectively, and one committer thread in all configurations. The x-axis of our worker-scaling experiments is the number of transaction worker threads; at 32 workers Ayame therefore runs 40 active threads (32 workers, 7 flushers, and one committer). At 96 workers Ayame runs 116 active threads (96 workers, 19 flushers, and one committer), which exceeds the 96 logical cores of the machine; we revisit this oversubscription when discussing the high-thread end of the sweep.

4.2 Experimental Configuration

4.2.1 Environment

Experiments were conducted on a server with two sockets of Intel(R) Xeon(R) Gold 5418N CPUs. Each socket had 24 physical cores with two hardware threads per core, for 48 physical and 96 logical cores in total, organized as two NUMA nodes. Each core had a 48 KiB private L1d cache and a 2 MiB private L2 cache, and each socket shared a 45 MiB L3 cache. The DRAM capacity was 256 GB. The machine ran Ubuntu 22.04.5 LTS (Linux kernel 5.15), and WAL files were placed on a locally attached SSD formatted with ext4. Worker threads were pinned to distinct logical cores with `sched_setaffinity`.

4.2.2 Benchmark and Workloads

All experiments used the Yahoo! Cloud Serving Benchmark (YCSB) [14]. We evaluated three representative workloads: YCSB-A (50% read, 50% update), YCSB-B (95% read, 5% update), and YCSB-C (read-only). These workloads exercise the WAL along a spectrum from update-heavy to read-only and are widely used to evaluate concurrency-control protocols. Each transaction issued a fixed number of ten operations over a database of 100,000 records with 4-byte values, and keys were drawn from a uniform distribution.

Table 1 Default parameter settings.

Type	Parameter	Setting
DB	#records	100,000
	value size	4 B
	key distribution	uniform
TX	#operations	10
	read ratio	50 / 95 / 100 %
Run	worker threads	1,2,4,8,16,32,48,96
	measurement time	5 s
	repeats	5
Ayame	flush group size [commits]	16
	flush interval	50 μs
	max pending acks	65,536

4.2.3 Metric

The primary metric is *durable-acknowledgment throughput*: the number of transactions acknowledged as durable to the client, divided by the measurement time. All systems are measured at the same stop barrier, and acknowledgments drained during shutdown are excluded. For the worker-wait systems (Single WAL and P-WAL) a logical commit and a durable acknowledgment coincide, so the metric has the same meaning across the three systems. We additionally report the p99 durable acknowledgment latency and the number of pending (not-yet-acknowledged) durable commits. Unless otherwise stated, each reported value is the mean of the corresponding per-run metric over five runs; reported p99 latencies are means of the per-run p99 values.

4.3 Results

We evaluate Ayame using the worker-thread sweep of Table 1, which spans 1 to 96 worker threads (the full two-socket machine). At 32 worker threads on YCSB-A, Single WAL achieves 8K durable acknowledgments per second and P-WAL achieves 58K, whereas Ayame achieves 253K durable acknowledgments per second. On YCSB-B, Single WAL achieves 25K and P-WAL achieves 200K, whereas Ayame achieves 442K durable acknowledgments per second. These results correspond to 4.4× and 2.2× improvements over P-WAL on YCSB-A and YCSB-B, respectively. Ayame also keeps the number of pending durable commits small at 32 worker threads: 1,657 pending commits on YCSB-A and 75 pending commits on YCSB-B. Across the upper half of the sweep the Ayame-over-P-WAL ratio on YCSB-A is stable, staying between 4.3× and 4.5× from 32 to 96 workers. Ayame's absolute YCSB-A throughput peaks near 32–48 workers (253K and 250K) and then declines at 96 workers (176K): Ayame's background flushers and committer raise the active thread count to 116, which oversubscribes the 96 logical cores. P-WAL declines in parallel over this range, so Ayame's relative improvement persists. At 4 and 8 worker threads on YCSB-A, however, pending durable commits approach the configured backpressure bound and the p99 acknowledgment latency rises into the hundred-millisecond range (Figures 3 and 4). This spike occurs because Ayame provisions only one or two flusher threads in these configurations, so the pipeline itself becomes the bottleneck un-

til additional flushers are introduced at 16 and 32 worker threads.

Figure 2 shows the worker-thread comparison. The results show that Ayame improves update and mixed read-write workloads, where durable logging is still required on every update commit. We also evaluate YCSB-C, where all transactions are read-only and WAL records are skipped. In this case, durability work largely disappears; after using the same binary and a read-only empty-frontier fast path, Single WAL, P-WAL, and Ayame perform similarly at 32 worker threads. We therefore use YCSB-C as a sanity check for read-only bookkeeping overhead rather than as evidence of WAL persistence performance.

Ayame's timestamp integration is primarily a design simplification: it removes duplicated logical ordering between concurrency control and WAL durability by reusing SSN's commit timestamp (cstamp) as the logical recovery order. We do not rely on timestamp integration alone as a direct throughput optimization. The main throughput effect comes from parallel WAL shards, batching, and separating workers from durable-wait processing; dependency-closed acknowledgment primarily prevents the pending-ack backlog and tail-latency blowups caused by global-prefix waiting (Section 4.4).

Table 2 summarizes the main 32-worker end-to-end results used in the figures. Table 3 extracts the Ayame-vs-P-WAL speedups from the same measurements.

4.4 Acknowledgment-Policy Comparison

The main end-to-end comparison combines several changes: timestamp-integrated ordering, asynchronous flushing, and dependency-closed acknowledgment. To isolate the effect of the acknowledgment rule itself, we run an acknowledgment-policy comparison that uses the same worker/flusher/committer pipeline as Ayame and changes only the condition for returning durable acknowledgments. The *global-prefix* variant acknowledges a transaction only after a single global durable prefix covers its logical commit position; the *dependency-frontier* variant acknowledges a transaction once the shard positions in its dependency frontier are durable.

The benefit of dependency-closed acknowledgment is not that a transaction has many dependencies. Its benefit appears when a transaction does *not* depend on some slow or lagging WAL shards: global-prefix acknowledgment still waits for those shards, whereas dependency-frontier acknowledgment does not.

Figure 6 and Table 4 report the result. With only one or two flusher threads (4 and 8 workers) both variants are limited by the pipeline itself. Once enough flusher threads are provisioned (16 workers and above), the difference emerges. Even without injecting an artificial straggler, the global-prefix variant accumulates a large backlog—its pending count stays near the configured backpressure limit—because natural progress variation among the WAL shards is enough to delay the global durable prefix. In contrast, dependency-frontier acknowledgment does not wait for unrelated shards and therefore keeps the pending durable commits lower, draining them further as more flusher threads are added.

The effect strengthens as flusher threads are added. On YCSB-B, where updates are light, the dependency-frontier rule drains almost completely from 16 workers upward: it holds fewer than 100 pending durable commits and about 1 ms p99 acknowledgment latency, whereas the global-prefix variant stays near the 65,536 backpressure limit with a 131 ms p99 (at 32 workers, 69 versus 65,470 pending commits and 0.5 ms versus 131 ms). On YCSB-A, whose update load is ten times higher, both variants saturate with only one or two flushers (4 and 8 workers); as flushers are added the dependency-frontier rule drains progressively—about 28,000 pending commits at 16 workers, 14,500 at 32, and 306 at 96—and its p99 falls to 2 ms, while the global-prefix variant stays saturated near 65,500 with a 262 ms p99 at every worker count. (Pending counts here are the measurement-window snapshot, as in Table 2; these acknowledgment-policy runs are separate from the main worker-scaling runs, so values differ slightly due to run-to-run variation.) Dependency-closed acknowledgment does not primarily improve short-window throughput in this comparison: at 32 workers the global-prefix variant even reaches slightly higher throughput on YCSB-A (238K versus 225K durable acknowledgments per second), while the dependency-frontier variant is slightly higher on YCSB-B (457K versus 395K). Its benefit is not primarily throughput; rather, it keeps the pending durable commits and the tail latency from exploding, while throughput remains comparable and sometimes lower. Because both variants share the same pipeline, this difference isolates the effect of the acknowledgment rule (**A3**) rather than the asynchronous batching.

We next ask whether this benefit requires dense dependency frontiers. Zipfian skew does not answer this question: it changes which keys are hot but not which WAL shards wrote them, so in a non-partitioned workload the writers of the accessed versions stay spread across all shards and the dependency frontier remains dense. We therefore vary the frontier width directly with an abort-free partitioned workload in which each worker owns a key partition and a remote-access probability controls how often a transaction touches other partitions. With `remote` = 0 every transaction depends only on its local shard—a sparse frontier of one shard per transaction (nz/tx = 1)—and with remote accesses the frontier quickly becomes dense (about seven shards). Because this abort-free workload commits—and therefore logs—on every transaction, its per-second WAL volume is roughly twice that of the mixed YCSB workloads, which skip read-only transactions. We therefore enlarge the flush group to 64 commits in this experiment so that the flush pipeline keeps up for *both* acknowledgment policies; this removes flush-pipeline saturation as a confound, so the remaining difference reflects only the acknowledgment rule. The effect is robust to this value: flush groups from 64 to

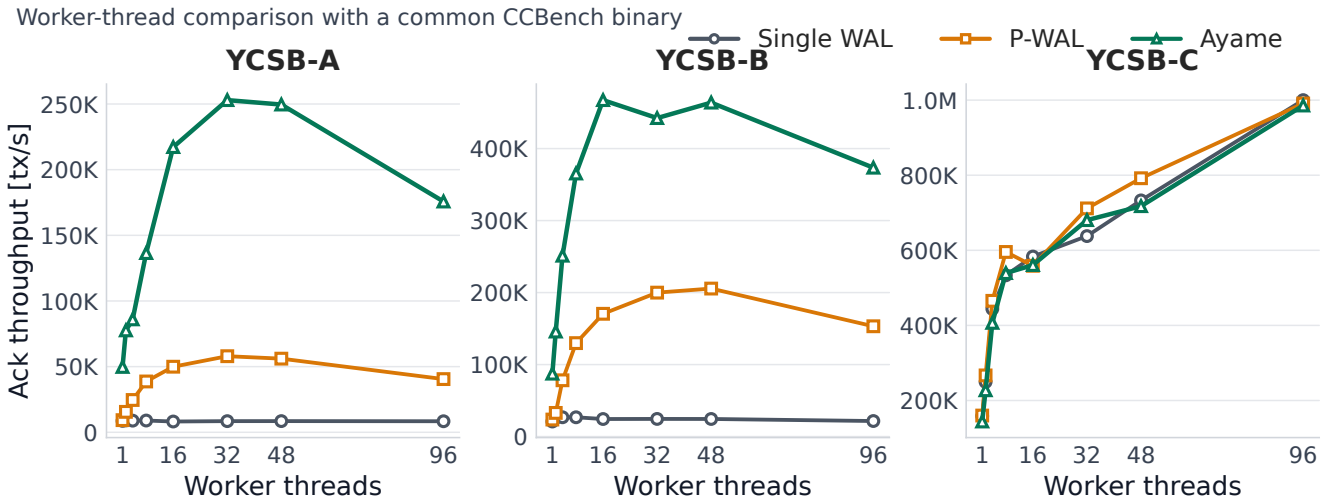

Fig. 2 Durable-acknowledgment throughput by transaction worker threads. Ayame improves update and mixed read-write workloads by combining parallel WAL logging, dependency-closed durable acknowledgment, and timestamp-integrated logical ordering.

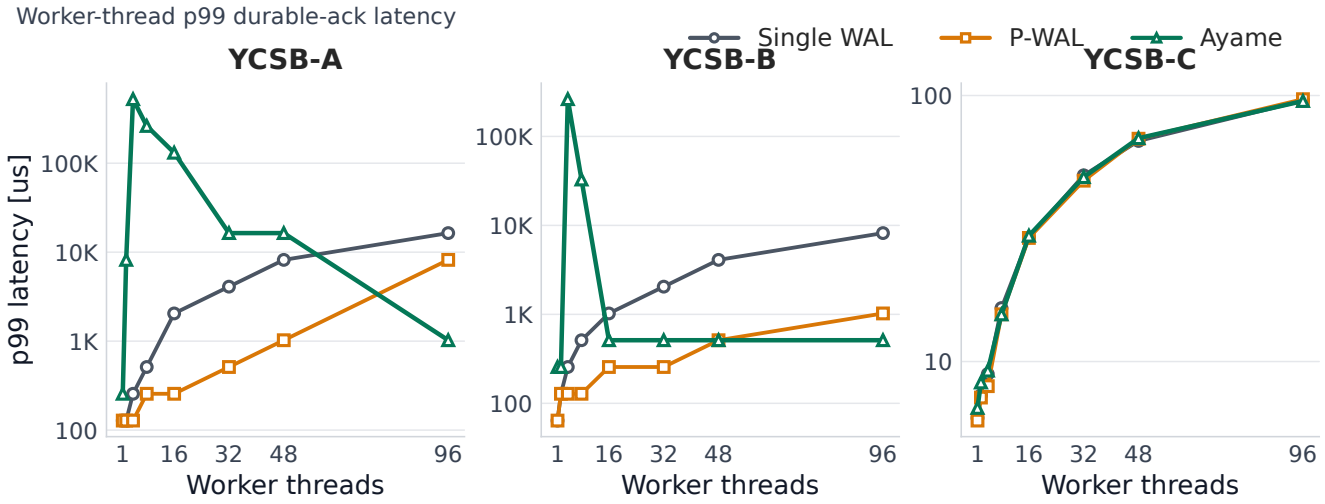

Fig. 3 P99 durable-acknowledgment latency by transaction worker threads.

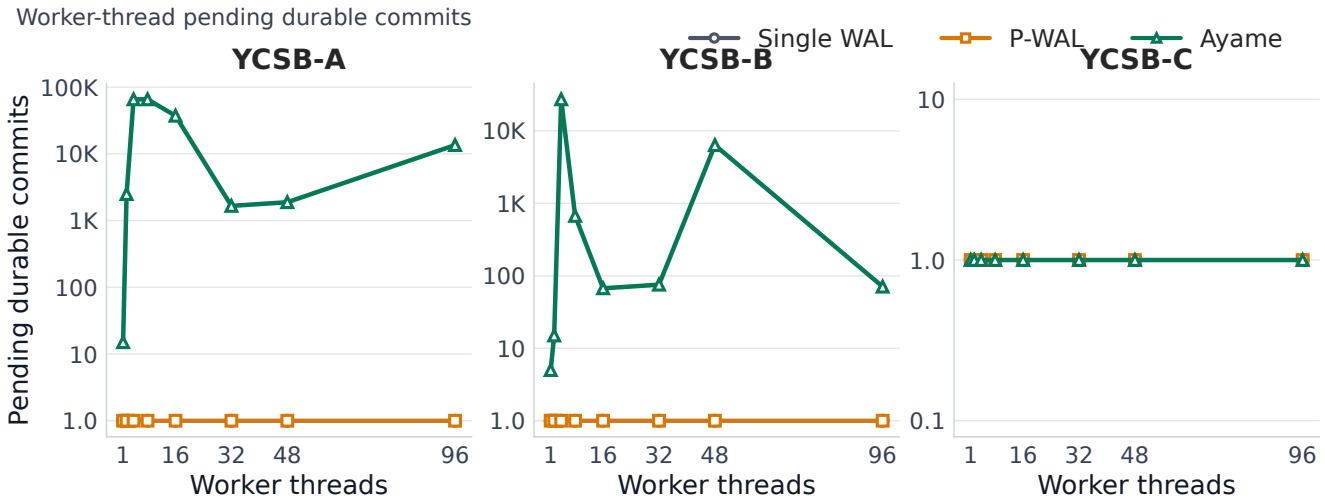

Fig. 4 Pending durable commits by transaction worker threads (log scale). Single WAL and P-WAL are synchronous and keep no pending durable commits; they are drawn at the axis floor.

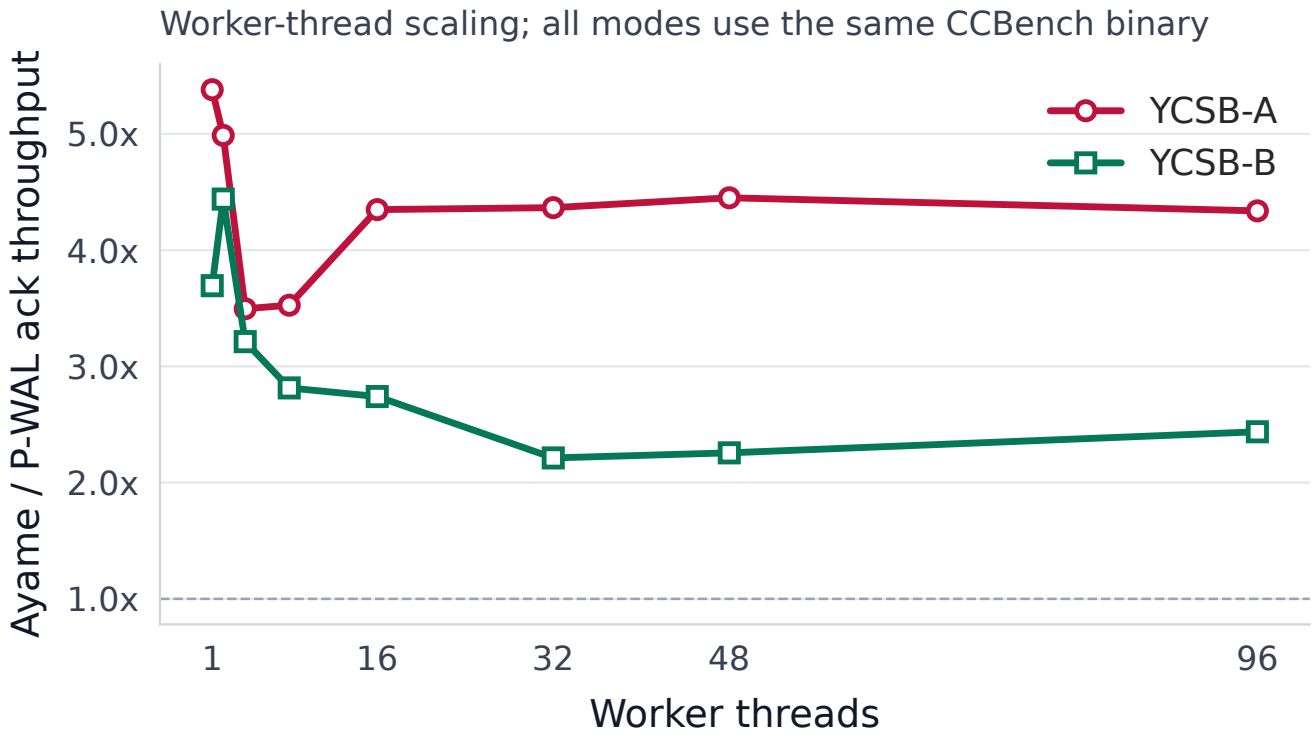

Fig. 5 Ayame throughput relative to P-WAL by transaction worker threads.

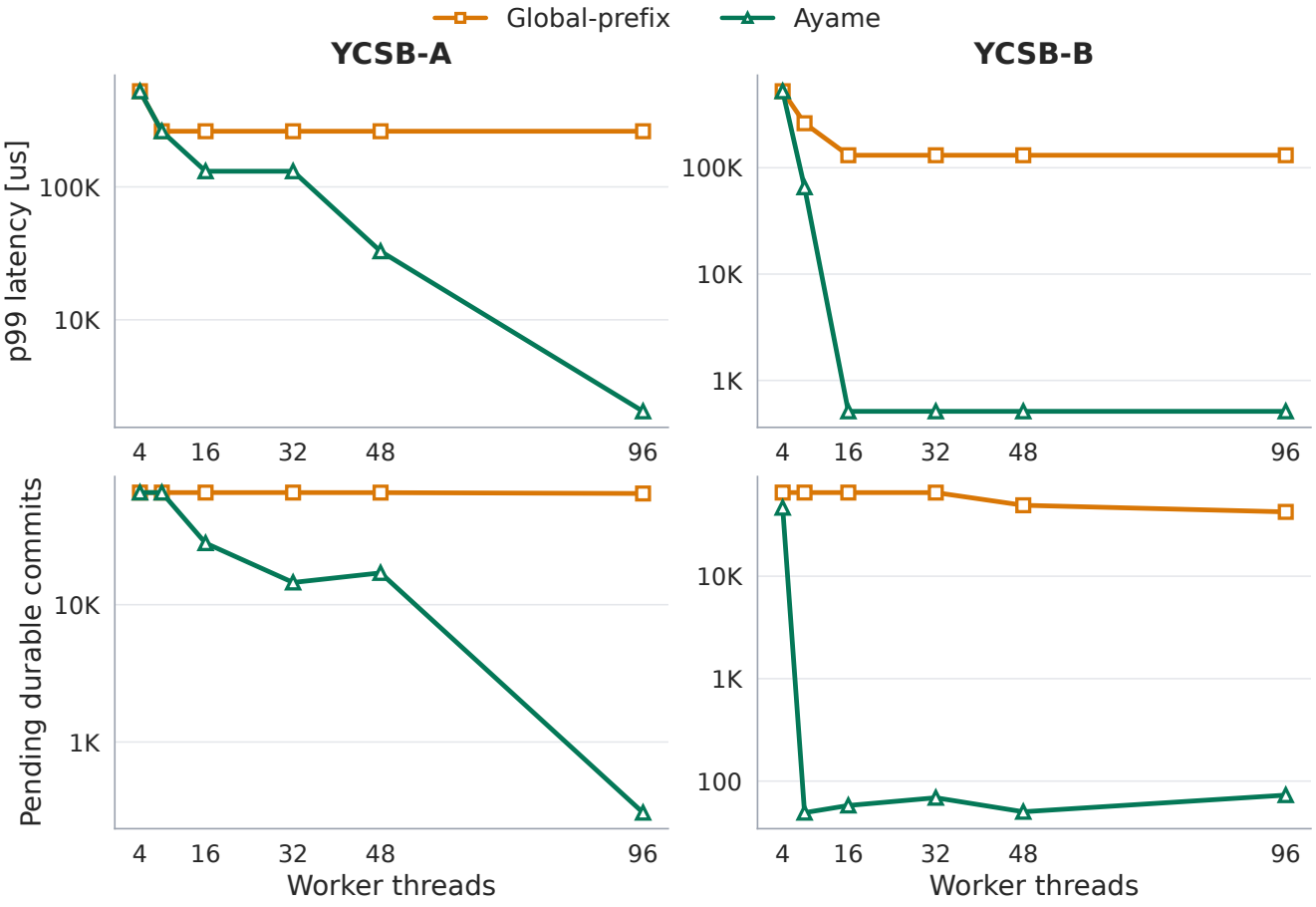

Fig. 6 Acknowledgment-policy comparison. Both variants use the same asynchronous WAL pipeline and differ only in the acknowledgment condition. The global-prefix rule keeps the pending count and p99 acknowledgment latency saturated, while the dependency-frontier rule drains them once enough flusher threads are provisioned.

Table 2 End-to-end YCSB results at 32 transaction worker threads.

Workload	System	Ack tps	p99 μs	Pending	fdatasync calls	Tx/sync	Frontier B/tx	WAL atomic/tx
YCSB-A	Single WAL	8.5K	4,096	0	42,451	1.0	0.0	6.00
YCSB-A	P-WAL	57.9K	512	0	289,466	1.0	0.0	6.00
YCSB-A	Ayame	253.0K	16,384	1,657	85,424	14.8	56.0	0.00
YCSB-B	Single WAL	24.5K	2,048	0	49,310	2.5	0.0	0.90
YCSB-B	P-WAL	199.9K	256	0	400,993	2.5	0.0	0.90
YCSB-B	Ayame	442.4K	512	75	105,638	21.3	56.0	0.00
YCSB-C	Single WAL	637.9K	50	0	0	0.0	0.0	0.00
YCSB-C	P-WAL	711.6K	48	0	0	0.0	0.0	0.00
YCSB-C	Ayame	680.4K	49	0	0	0.0	56.0	0.00

Ack tps is durable-acknowledgment throughput. Ayame uses 7 flusher threads and 1 committer thread at 32 workers.

Table 3 Ayame throughput relative to P-WAL at 32 transaction worker threads.

Workload	P-WAL tps	Ayame tps	Speedup	Ayame p99 μs	Ayame pending
YCSB-A	57.9K	253.0K	4.37×	16,384	1,657
YCSB-B	199.9K	442.4K	2.21×	512	75
YCSB-C	711.6K	680.4K	0.96×	49	0

Table 4 Acknowledgment-policy comparison at 32 worker threads. Both policies use the same asynchronous pipeline and differ only in the acknowledgment condition.

Workload	Ack policy	Ack tps	p99 μs	Pending
YCSB-A	Global-prefix	237.7K	262,144	65,524
YCSB-A	Dep. frontier (Ayame)	225.4K	131,072	14,509
YCSB-B	Global-prefix	394.7K	131,072	65,470
YCSB-B	Dep. frontier (Ayame)	457.5K	512	69

Table 5 Dependency-density experiment at 32 worker threads on an abort-free partitioned workload. The remote-access probability sets the frontier width (nz/tx is the mean number of shards per dependency frontier).

Remote	nz/tx	Ack policy	Ack tps	p99 μs	Pending
0%	1.0	Global-prefix	353.4K	131,072	65,370
0%	1.0	Dep. frontier (Ayame)	269.6K	2,048	264
1%	6.9	Global-prefix	350.5K	131,072	65,409
1%	6.9	Dep. frontier (Ayame)	259.8K	1,024	28
10%	7.0	Global-prefix	347.5K	131,072	65,37
10%	7.0	Dep. frontier (Ayame)	278.2K	2,048	30
100%	7.0	Global-prefix	351.3K	131,072	65,41
100%	7.0	Dep. frontier (Ayame)	269.4K	2,048	27

1,024 all leave the dependency-frontier queue drained while the global-prefix queue stays saturated.

Table 5 reports the result. The global-prefix rule essentially saturates the pending-ack queue and holds a 131 ms p99 at every density, including the fully sparse case, because it waits for the global durable prefix regardless of what a transaction actually depends on. The dependency-frontier rule keeps the pending count to a few hundred commits and the p99 to about 2 ms across the whole range; its throughput is again slightly lower, as in the comparison. Crucially, even when each transaction depends on a single shard (nz/tx = 1), the dependency-frontier rule still keeps pending durable commits and tail latency far below the global-prefix rule: about 260 versus 65,000 pending commits, and 2 ms versus 131 ms p99. The value of dependency-closed acknowledgment is therefore not that a transaction has many dependencies, but that it does not wait for the shards it does not depend on, and this value remains when frontiers are sparse.

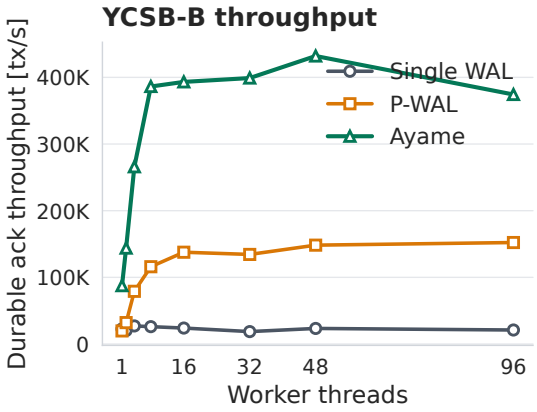

Fig. 7 YCSB-B durable-acknowledgment throughput in the perf-stat worker-scaling runs.

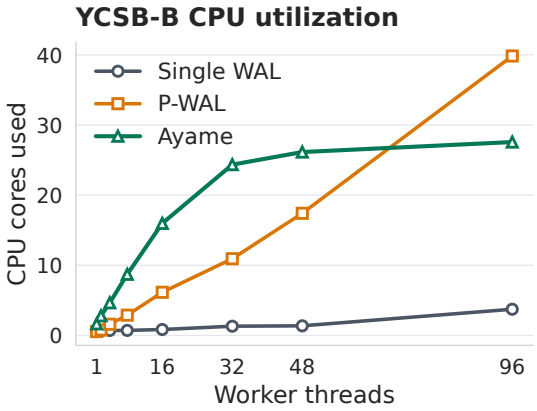

Fig. 8 CPU-core utilization in the YCSB-B perf-stat worker-scaling runs.

4.5 Hardware Counters and Overhead

Table 6 summarizes the 32-worker perf-stat analysis. Table 7 shows the worker-scaling perf-stat data for YCSB-B, where Ayame's throughput improvement is most visible. The perf-stat runs are separate from the main throughput runs; their absolute throughputs therefore differ slightly due to profiling overhead and run-to-run variation.

Finally, Table 8 reports the top self-time symbols for YCSB-C. Since YCSB-C is read-only, this table is used as a sanity check that the remaining cost is read-path bookkeep-

Table 6 Perf-stat summary at 32 transaction worker threads.

Workload	System	Ack tps	CPU cores	Cycles/tx	Instr/tx	IPC	Ctx sw.	Tx/sync
YCSB-A	Single WAL	7.5K	1.3	472,527	755,683	1.62	125,815	1.0
YCSB-A	P-WAL	42.6K	10.2	609,702	517,618	0.85	1,119,845	1.0
YCSB-A	Ayame	147.6K	18.6	315,832	374,322	1.17	1,431,732	15.4
YCSB-B	Single WAL	18.4K	1.3	254,884	207,058	1.00	118,337	2.5
YCSB-B	P-WAL	124.3K	9.0	213,495	158,618	0.79	956,930	2.5
YCSB-B	Ayame	507.7K	23.3	138,704	121,004	0.78	2,377,111	34.8
YCSB-C	Single WAL	654.7K	32.4	128,525	29,586	0.23	3,126	0.0
YCSB-C	P-WAL	629.1K	32.5	133,945	30,034	0.22	3,405	0.0
YCSB-C	Ayame	660.0K	32.5	127,754	33,304	0.26	34,728	0.0

Table 7 YCSB-B perf-stat worker scaling.

Workers	Single tps	P-WAL tps	Ayame tps	Single CPU	P-WAL CPU	Ayame CPU
1	22.7K	19.6K	87.5K	0.6	0.6	1.7
2	20.3K	32.1K	143.9K	0.7	0.8	2.8
4	27.1K	79.1K	265.9K	0.7	1.6	4.7
8	26.1K	116.0K	386.3K	0.7	2.9	8.7
16	24.0K	137.7K	393.2K	0.9	6.2	16.0
32	18.7K	134.4K	399.0K	1.3	10.9	24.3

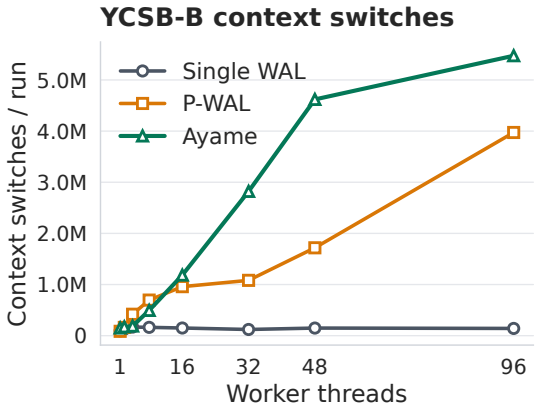

Fig. 9 Context-switch counts in the YCSB-B perf-stat worker-scaling runs.

ing rather than WAL persistence.

5. Related Work

6. Conclusion

Acknowledgments

References

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[5] Ports, D. R. K. and Grittner, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, pp. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).

[6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).

[7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium*

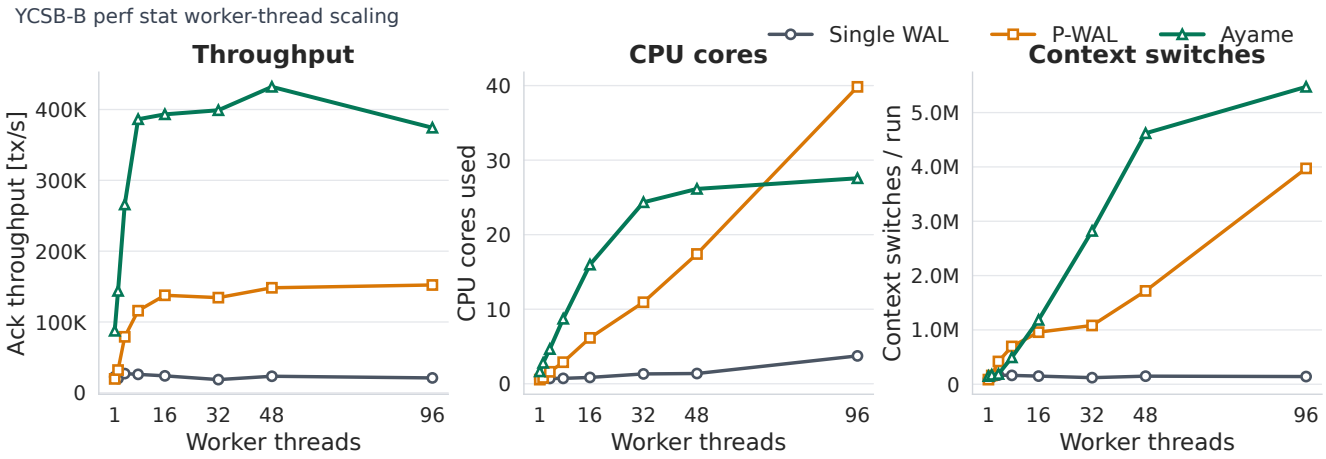

Fig. 10 Perf-stat summary for YCSB-B worker scaling.

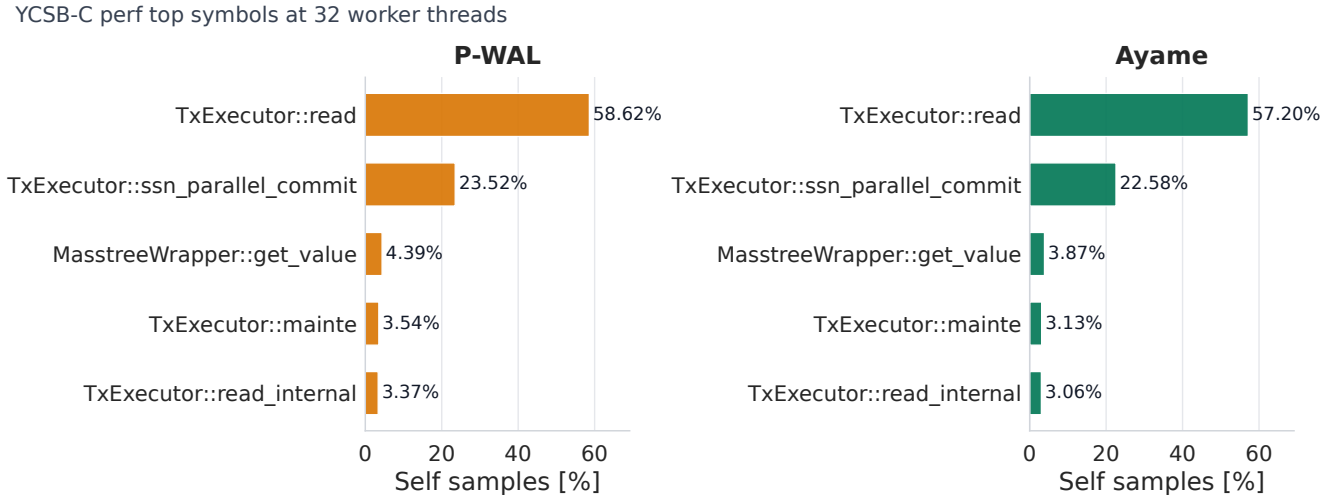

Fig. 11 Top self-time symbols for YCSB-C at 32 transaction worker threads.

Table 8 Top self-time symbols for YCSB-C at 32 transaction worker threads.

System	Rank	Self %	Symbol
Single WAL	1	58.81	`TxExecutor::read`
Single WAL	2	24.00	`TxExecutor::ssn_parallel_commit`
Single WAL	3	4.41	`MasstreeWrapper::get_value`
Single WAL	4	3.31	`TxExecutor::mainte`
Single WAL	5	3.02	`TxExecutor::read_internal`
P-WAL	1	58.62	`TxExecutor::read`
P-WAL	2	23.52	`TxExecutor::ssn_parallel_commit`
P-WAL	3	4.39	`MasstreeWrapper::get_value`
P-WAL	4	3.54	`TxExecutor::mainte`
P-WAL	5	3.37	`TxExecutor::read_internal`
Ayame	1	57.20	`TxExecutor::read`
Ayame	2	22.58	`TxExecutor::ssn_parallel_commit`
Ayame	3	3.87	`MasstreeWrapper::get_value`
Ayame	4	3.13	`TxExecutor::mainte`
Ayame	5	3.06	`TxExecutor::read_internal`

YCSB-C is read-only; WAL bytes and fdatasync counts are zero for all systems.

on Operating Systems Design and Implementation, OSDI '14, USENIX Association, pp. 465–477 (online), available from ⟨https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting⟩ (2014).

[11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from ⟨https://www.vldb.org/pvldb/vol7/p865-wang.pdf⟩ (2014).

[12] 神谷, 孝., 川島, 英., 星野, 喬., 建部, 修., Komei, K., Hideyuki, K., Takashi, H. and Osamu, T.: P-WAL: Paralell Write-ahead Logging, **情報処理学会論文誌データベース（*TOD*）**, Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[13] Kawashima, H.: Next-generation Database Tsurugi: Concurrency Control and Recovery Systems, *IPSJ Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).

[14] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).